

CERTIFICATION PROGRAM IN APPLIED GENERATIVE AI AND AGENTIC AI

Customer Review Insights & Response Generator

Final Project Report

A multilingual pipeline that collects product reviews in six languages, translates them, measures sentiment, finds the issues customers complain about, spots sudden spikes, and drafts on-brand replies for a human to approve.

Group 5

TalentSprint capstone project, final presentation on Sunday 11 October 2026

Code repository: github.com/west01srinivas-hue/Capstone_Project_Group_5

Team

Group 5 has seven members. Srinivasan K and Jyothi Kawalli are the group coordinators.

Member	
Srinivasan K	
Jyothi Kawalli	
Shishir Rai	
Shyam	
Sai Vamsi Krishna Posani	
Vinay Bommireddy	
Bandi Shivani	

How this report is organised

The sections follow the order of the final presentation so the two documents can be read side by side: problem, proposed solution, components, methods tried and challenges, demo plan, limitations, future scope and the business view. Section 10 explains how to run the application and Section 8 holds the test cases and results. Every number in the report comes from the experiment log in the repository (`docs/experiment_log.md`); where a result is weaker than the proposal's target, the report says so.

Contents

Team	2
How this report is organised	2
1. Executive summary.....	5
Results against the targets	5
2. Problem statement.....	5
Current methods and where they fail	6
3. Objectives and success metrics	6
4. Proposed solution and architecture	7
Design principles.....	7
Tools and technology	8
5. Data and pipeline design	8
5.1 Dataset.....	8
5.2 Unified schema	8
5.3 Samples and splits	8
5.4 Ground truth and its limits	9
6. Components in detail	10
6.1 Ingestion (n8n workflow 01)	10
6.2 Translation (n8n workflow 02)	10
6.3 Sentiment, topic and estimated stars	10
6.4 Issue clustering	11
6.5 Trend detection and alerts	12
6.6 Response generation and quality checks	13
6.7 Dashboard and Slack alerts	14
6.8 API service and LLMOps.....	14
6.9 The full pipeline in n8n (workflow 03).....	15
6.10 Demo app and deployment.....	15
7. Methods tried, challenges and how we overcame them.....	16
8. Test cases, results and experiment log.....	17
8.1 Automated tests	17
8.2 Functional test cases	17
8.3 Experiment log summary.....	17
8.4 Claude spend	18
9. Demo plan	18
10. How to run the application.....	20
10.1 Setup (Windows)	20
10.2 n8n workflows	20
10.3 Analysis scripts.....	20
10.4 API service and tests.....	20

10.5 Demo app and Hugging Face Space	21
10.6 Repository map	21
11. Limitations	21
12. Future scope and scaling	22
13. Business perspective	22
Illustration of the saving.....	22
14. Conclusion	23
References	23
Appendix A. Prompts and versions.....	24
Appendix B. Dashboard screenshots.....	24

1. Executive summary

Companies that sell in several countries receive reviews in many languages. Reading them by hand is slow, so problems such as late deliveries or damaged packaging are found after many customers have already been upset, and replies are late, inconsistent or missing. Our project builds an automated pipeline that reads reviews in English, German, Spanish, French, Japanese and Chinese, translates them, scores sentiment, groups negative reviews into named issue categories, alerts the team when one issue suddenly grows, and drafts a reply in the customer's own language.

We built and tested every stage with Claude Haiku 4.5 as the only language model for the pipeline and Claude Sonnet 5 as an independent judge of reply quality. The workflow runs in n8n, the analysis code and the API service run in Python, results are shown in a Google Sheets dashboard, and spike alerts go to Slack. A public demo app on a Hugging Face Space lets anyone try the pipeline. The whole project used \$4.29 of Claude credit (Anthropic Console total), within the \$5 budget.

Results against the targets

Target (from the brief and proposal)	What we measured	Status
1,000+ reviews per day	n8n Cloud processed 120 reviews in 81 s (0.68 s per review), so about 11 minutes per 1,000 reviews. Cost about \$2.50 per 1,000 reviews. Measured on 120 reviews and extrapolated.	Met on speed
85%+ sentiment accuracy	Positive vs negative: 88.4% on 960 held-out reviews. Three-way (positive, neutral, negative): 78.1% .	Two-way met, three-way not met
Response in 3 hours instead of 24	A draft is ready about a minute after a batch starts. Negative replies then wait for a human, and that waiting time was not measured.	Automation met, human time not measured
Star rating error (MAE) at most 0.15	MAE 0.47; 95.6% of estimates are within one star. The target is not realistic for this data (Section 6.3).	Not met
80% of drafts accepted	79.2% accepted by the Sonnet 5 judge (v2 prompt). No human acceptance test was run.	Close, on a proxy measure
Trend spikes found early	On simulated weekly data: precision 88.6%, recall 94.3%, about 0.5 false alarms per 12 weeks. Slack alert verified end to end.	Met on simulated dates
Issue clustering	13 readable issue categories; silhouette 0.07 (0.04 when clustering native text).	Working, low absolute score

What the results mean. The pipeline works end to end and is cheap and fast enough for the target volume. Its weak point is the neutral, three-star class, where even a strong language model agrees with the star rating only about a third of the time, because a three-star review is often genuinely mixed. We report this honestly instead of tuning until the number looked better.

2. Problem statement

An online seller with customers in several countries gets thousands of reviews a week in different languages. The team that has to act on them faces four connected problems:

- **Language barrier.** Reviews in Japanese, Chinese or German are skipped or copied one by one into a translator.
- **Stars hide the reason.** An average of 3.8 stars does not say whether customers are unhappy with delivery, price or quality.

- **Spikes are found late.** A week of damaged parcels shows up in a monthly report, after hundreds of customers have complained.
- **Replies are slow and generic.** Support answers in about a day, with template text that ignores what the customer actually wrote.

Current methods and where they fail

Situation (example)	How it is handled today	Where it fails
A three-star English review: "I love the sound but it turns itself back on."	Counted as a neutral star rating in an average.	The review contains a clear product fault (the device restarts itself) that nobody sees in the average.
A review saying the parcel "arrived late and the box was crushed".	A support agent reads it and replies by hand, or it is ignored.	Two separate issues (delivery and packaging) are lumped together, and the reply is late and generic.
A Japanese review of a product that is fine but has one annoying flaw.	Pasted into a translator by someone who happens to read the language, or skipped.	Slow, not repeatable, and the reply is written in the wrong language or not at all.
A week in which many customers report delayed shipments.	Discovered in a monthly summary or when the volume of complaints becomes obvious.	The alert comes weeks late; nobody compares this week with the recent baseline.

These four situations are the ones our demo (Section 9) goes back to, so that the audience can see each problem and the pipeline's answer to it.

3. Objectives and success metrics

The project brief asks for a working prototype of the workflow (aggregation, translation, analysis, response generation), a detailed report, and dashboard screenshots with a business-impact summary. Our objectives and how we check them:

Objective	How we measure it	Target
Aggregate reviews from several languages into one schema	1,200-review sample: 200 per language, 240 per star rating, unique ids	All six languages
Translate non-English reviews	Valid JSON, ids match, language detected correctly, no failed reviews	No failures
Classify sentiment and topic	Accuracy and macro-F1 against star-derived labels on 960 held-out reviews	85%+
Group negative reviews into issues	Readable cluster labels; silhouette score; agreement with Claude labels	Coherent, cross-language clusters
Detect sudden spikes	Precision, recall and false alarms on planted spikes over 200 simulated timelines	High precision, few false alarms
Draft replies safely	Judge scores (empathy, specificity, correctness, tone), guardrail flags, share accepted	80% accepted, no promises or wrong language
Show results and alert the team	Google Sheets dashboard; Slack message posted for each spike	Working end to end
Stay within budget and be repeatable	Cost per 1,000 reviews, spend cap, every call logged	Within \$5 total

4. Proposed solution and architecture

The solution is a pipeline of small steps. Each step reads the output of the one before it, records what it did, and marks a review as failed instead of stopping the whole run when something goes wrong.

n8n workflow 03 (25 nodes): Claude Haiku 4.5 does every language step

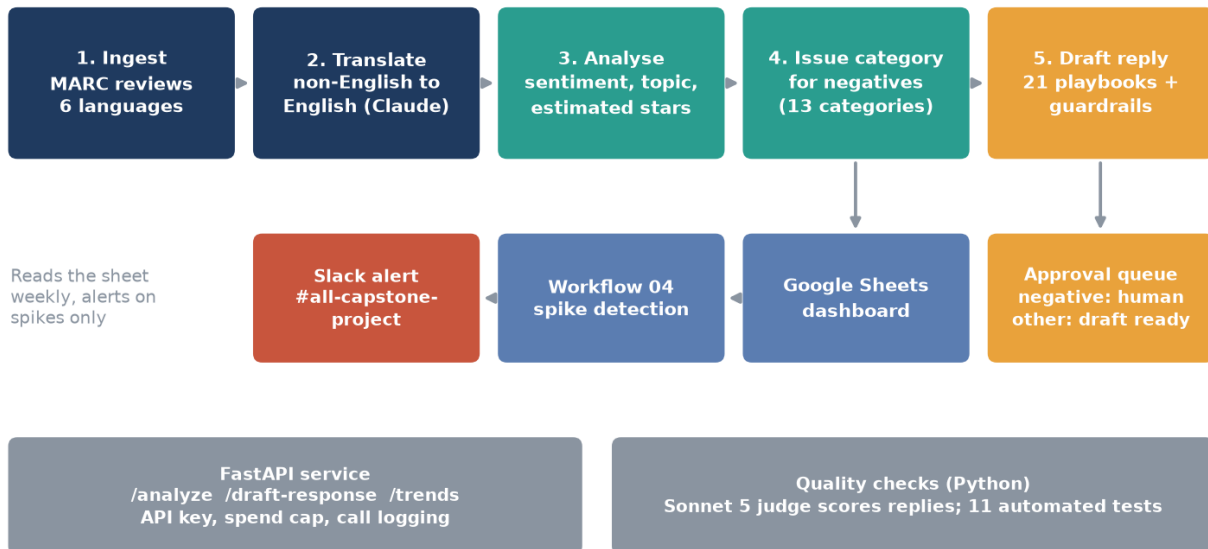


Figure 1. Architecture: the n8n pipeline (top), outputs (middle) and supporting services (bottom).

Design principles

- **One model family.** Claude Haiku 4.5 does translation, classification, issue labelling and drafting because it is fast and cheap (\$1 per million input tokens, \$5 per million output). A different, stronger model (Sonnet 5) judges the drafts so that a model does not grade its own work.
- **Human in the loop.** A reply to a negative review is never sent automatically. It always goes to a human approval queue. Neutral and positive replies are marked ready only if the judge accepts them.
- **Guardrails in the prompt.** Replies must be in the reviewer's language, use only facts from the review, promise no refund, replacement, discount or date, make no legal admissions, request no personal data and stay within 40 to 90 words.
- **Versioned prompts and honest measurement.** Every prompt is a file in prompts/ with a version number. Settings are chosen on a development split and reported once on a held-out test split.
- **Cost control.** The API service has a hard spend cap and logs every call with tokens, cost and latency.

Tools and technology

Purpose	Tool	Why
Workflow automation	n8n Cloud (workflows exported as JSON)	Visual workflow, retries, easy to demo; required by the brief
Language model	Claude Haiku 4.5 (pipeline), Claude Sonnet 5 (judge)	Strong multilingual quality at low cost
Data	Multilingual Amazon Reviews Corpus (Kaggle copy)	Real reviews in six languages with star ratings
Embeddings and clustering	sentence-transformers, k-means (scikit-learn)	Free, runs locally, multilingual
Baseline model	TF-IDF character n-grams + logistic regression	Shows what an LLM adds over a classic model
Service and tests	FastAPI, pytest	Three endpoints plus 16 automated tests
Demo app and hosting	Gradio on a Hugging Face Space; GitHub	Public three-tab demo with a spend cap; code and run instructions in the repository
Dashboard and alerts	Google Sheets, Slack incoming webhook	Free, familiar to business users

5. Data and pipeline design

5.1 Dataset

We use the Multilingual Amazon Reviews Corpus (MARC, Keung et al., EMNLP 2020): real Amazon product reviews in English, German, Spanish, French, Japanese and Chinese, with a 1 to 5 star rating, a product category and a review text (title and body). The team downloaded the Kaggle copy (mexwell/amazon-reviews-multi). We checked it against the Hugging Face copy used by the n8n Cloud workflows: all 1,200 sample reviews have identical ids, star ratings and text, so results from either copy are interchangeable.

5.2 Unified schema

```
review_id, source, product_category, rating, review_text,
detected_language, translated_text, review_date
```

5.3 Samples and splits

Set	Size	Use
Main sample	1,200 reviews (40 per star per language)	All evaluation; stratified because the source files are sorted by star rating
Development split	240 reviews	Choosing prompts (prompt v1 vs v2)
Test split	960 reviews	Reported once; never used for tuning or training
Stacker training sample	600 reviews from the MARC train split (100 per language)	Training the optional stacking model; the baseline never saw it
Baseline training data	30,000 reviews from the validation file	TF-IDF + logistic regression, one model per language
Dashboard set	1,560 analysed reviews (test split plus train sample)	Google Sheets dashboard and trend workflow

5.4 Ground truth and its limits

MARC has no human sentiment labels, so we derive them from stars: 1 to 2 stars is negative, 3 is neutral, 4 to 5 is positive. This is a noisy proxy, and it limits how high any text model can score (Section 6.3). The dataset also has no review dates, so the trend stage uses simulated dates (12 weeks from 6 July 2026) with planted spikes. Trend results therefore show that the detector works, not how real customers behave.

6. Components in detail

6.1 Ingestion (n8n workflow 01)

The workflow downloads the MARC review files, takes an even sample per star rating and language, and outputs the unified schema. It was verified by a headless run: 1,200 items, 200 per language, 240 per star, unique ids, no encoding damage in any language. The default run uses 4 reviews per star per language (about 100 translations) so that test runs cost pennies; a setting scales it up.

6.2 Translation (n8n workflow 02)

Non-English reviews are translated to English in batches of eight with Claude. The prompt returns JSON with the detected language and translation for each id. Each call is retried three times; a failed batch marks its reviews failed with the error and the run continues.

	Groq Qwen 27B	Claude Haiku 4.5
Valid JSON, ids match, language correct (40 reviews)	40/40	40/40
Latency per batch	1.5 s to 58 s (erratic)	5.6 s to 21.7 s
Cost	Free tier	\$0.0218 for 40 reviews (about \$0.54 per 1,000)

We chose Claude only. Both models translated all 40 test reviews correctly, but Groq's latency was unpredictable and Groq no longer offers the Llama chat models that our early plan assumed. Cloud and local runs of the workflow agree: 120 reviews, 100 translated, 20 English (not needed), 0 failed, about \$0.06. With an invalid API key the run still finishes and records all 100 as failed with the error message.

6.3 Sentiment, topic and estimated stars

One Claude call per review returns the sentiment, a topic (quality, features, shipping, price, ux, support or other), a confidence and an estimated star rating from 1 to 5. Three prompt versions were tried; version 3 adds the star estimate and is used in production, with the label taken from the estimated stars. We compared it with a classic baseline.

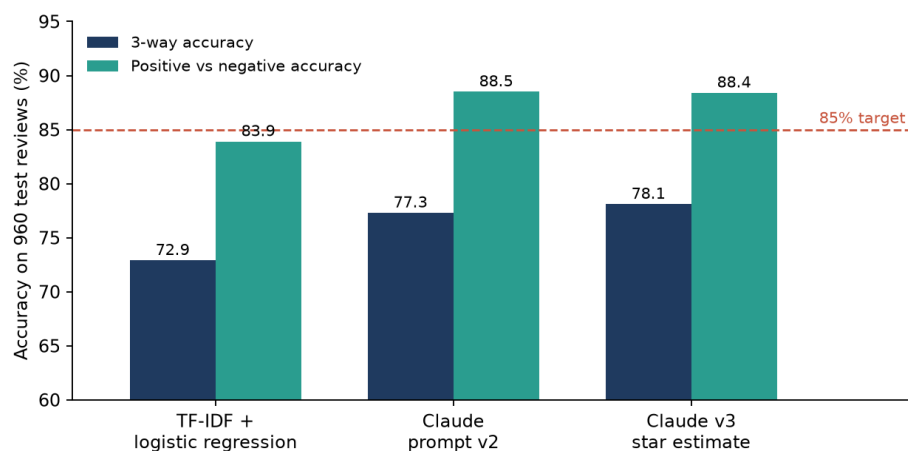


Figure 2. Sentiment accuracy on the 960 held-out test reviews.

Model (test, 960 reviews)	3-way accuracy	Macro-F1	Neutral recall	Positive vs negative
TF-IDF baseline	72.9%	0.654	29.2%	83.9%
Claude prompt v2	77.3%	0.697	32.3%	88.5%
Claude prompt v3, label	77.7%	0.707	35.4%	88.3%
Claude v3, star estimate to label (production)	78.1%	0.714	37.0%	88.4%
Stacker, Claude + baseline	78.1%	0.708	34.4%	89.1%
Stacker, balanced (optional mode)	75.4%	0.720	57.8%	79.8%

Confusion matrix for prompt v2 (rows are the star-derived truth):

Truth \ predicted	Negative	Neutral	Positive
Negative (384)	357	22	5
Neutral (192)	101	62	29
Positive (384)	17	44	323

Positive vs negative accuracy by language (prompt v2): English 89.1%, German 89.1%, Spanish 87.5%, French 95.3%, Japanese 89.1%, Chinese 81.2%. Chinese is the weakest language for Claude; the baseline is stronger there (84.4%).

Why three-way accuracy stays near 78%

- The gap is almost entirely the neutral class: 101 of 192 three-star reviews were called negative. Negative recall is 93% and positive 84%.
- A three-star review is often genuinely mixed (“I love the sound but it turns itself back on”). Star ratings are a noisy proxy for text sentiment, and the published fine-grained MARC results also stop at about 59 to 70% exact-star accuracy.
- We tried to lift the neutral class with a small logistic-regression stacker over Claude’s label, confidence, star estimate and the baseline’s probabilities. Everything landed at 75 to 78%. A “balanced” stacker finds more neutrals (recall up to 58%) but pushes borderline reviews out of the positive and negative classes, and positive-vs-negative accuracy falls from 88% to 80%, below the target. We therefore kept the simple, cheaper classifier and offer the balanced stacker only as an optional “flag borderline reviews” mode.
- Star estimate versus true stars: MAE 0.47, exact match 58.0%, within one star 95.6% (Chinese weakest, MAE 0.63). The proposal’s MAE target of 0.15 is not realistic for this data.

Topic labels were produced for every review but could not be checked against ground truth, because the data has none. The distribution on the test set was quality 536, features 199, shipping 104, price 49, other 37, ux 19, support 16.

6.4 Issue clustering

For the 765 reviews classified negative in the 1,560-review dashboard set, we compared two ways of grouping them. Method A embeds the native review text with a multilingual sentence model and clusters it. Method B first asks Claude for a short English “issue phrase” (what went wrong, in a few words), then embeds and clusters those phrases. This is the proposal’s “translate-then-analyze versus native” comparison.

Text that is embedded	k-means silhouette (k = 6 to 14)	HDBSCAN	Share of one topic within a cluster
Native review text, six languages	0.037 to 0.044	0 clusters, all noise	42 to 86%
Claude's English issue phrase	0.056 to 0.071 (best k = 13: 0.0706)	3 clusters, 86% noise	42 to 96%

Translate-then-analyze wins: the silhouette rises about 1.6 times and clusters mix languages evenly, which is the goal of grouping across languages. The absolute score (0.07) is low, as is normal for short phrases about very different products, so we treat it as a coarse measure and rely on the fact that the clusters read clearly. HDBSCAN produced almost only noise, so we use k-means with k = 13.

The 13 categories, named by Claude from the cluster contents: delayed or missing shipments and poor support; damaged packaging and product during shipping; wrong item received or non-functional; missing parts and assembly defects; product fails or breaks shortly after purchase; defective or malfunctioning components; poor overall quality and durability; connectivity and performance issues; design flaws and uncomfortable fit; wrong size sent or undersized; poor content quality and printing errors; product does not work as advertised; overpriced and rapid price drops.

n8n Cloud cannot run the embedding model, so inside n8n a Claude call assigns each negative review to one of these 13 categories. On 100 negatives its choice matched the k-means cluster 43% of the time. That is not an accuracy figure, since k-means is not ground truth; the disagreements were mostly reasonable re-assignments (for example, "misleading description" going to "does not work as advertised"). Only 2% of reviews got "none fits".

6.5 Trend detection and alerts

The detector counts negative reviews per issue category per week and compares each week with the trailing four weeks. A week is flagged when its z-score is at least 3.5 and it has at least 8 reviews. The spread is floored at the Poisson value (the square root of the mean) so that quiet categories do not flag on noise. An EWMA control chart is available as an alternative.

Because the data has no dates, we assigned each review a simulated week and planted three spikes: delayed shipments five times normal in week 8, damaged packaging four times in week 10 and price complaints three times in week 11 (weeks counted from 1). Results are averaged over 200 random timelines.

Rule	Precision	Recall	Precision@3	False alarms per 12 weeks
z-score (chosen)	88.6%	94.3%	91.3%	0.47
EWMA	90.6%	86.0%	91.3%	0.34

Threshold sweep for the z-score rule: z 3.0 gives recall 97% but precision 77% and 1.07 false alarms; z 4.0 gives recall 90.7%, precision 91.6% and 0.32 false alarms; z 5.0 gives recall 75% and precision 97%. These numbers are optimistic because the threshold was tuned on the same simulation. Precision@5, the proposal's metric, cannot exceed 0.6 with three planted spikes, so Precision@3 is reported.

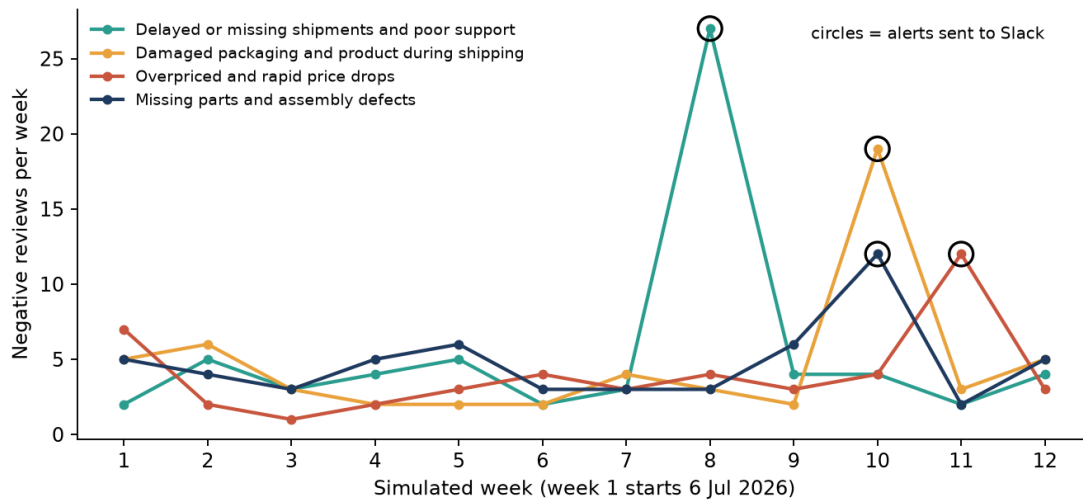


Figure 3. Weekly negative reviews for four issue categories in the dashboard data. Circles mark the four alerts sent to Slack.

The dashboard data produced four alerts: delayed shipments (27 reviews in the week of 24 August, baseline 3.5, z 12.56), damaged packaging (19 in the week of 7 September, z 9.80), price complaints (12 in the week of 14 September, z 4.54) and missing parts (12 in the week of 7 September, z 4.26). The first three are planted spikes. The fourth was not planted, so it is a false alarm produced by ordinary variation in a small category, which is consistent with the roughly 0.5 false alarms per 12 weeks measured above.

6.6 Response generation and quality checks

A library of 21 playbooks, one for every combination of sentiment (three) and issue type (seven), gives the drafting model the right tone and structure. A system prompt adds the brand voice and the hard guardrails listed in Section 4. One Claude Haiku call drafts each reply. The brand (“Acme Store”) and its support address are placeholders, and MARC has no reviewer names, so four synthetic first names per language test personalisation.

Quality is checked by Claude Sonnet 5 as an independent judge. It scores empathy, specificity, correctness and tone from 1 to 5 and flags promised outcomes, invented facts, wrong language and reviews a human must handle. A draft counts as judge-acceptable when all four scores are at least 4 and no flag is raised. We tested on 48 held-out reviews (8 per language: 4 negative, 2 neutral, 2 positive).

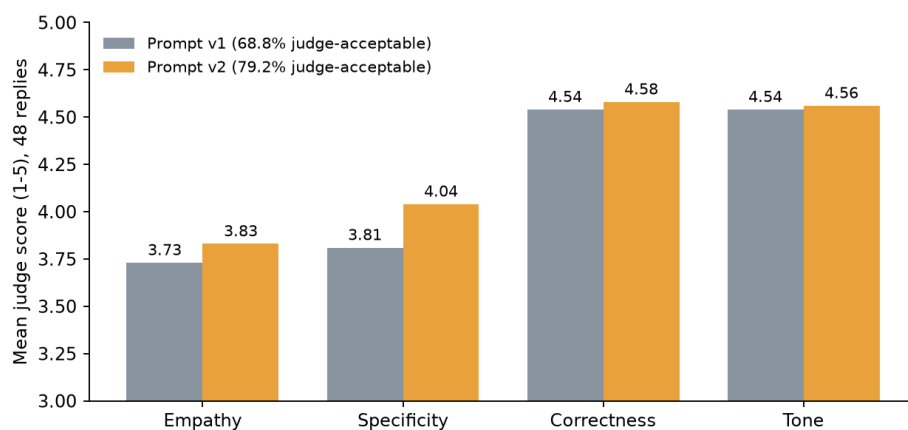


Figure 4. Reply quality, prompt v1 versus v2, scored by the Sonnet 5 judge on 48 replies.

Prompt	Promised outcome	Invented facts	Wrong language	Judge-acceptable
response v1	1	4	0	68.8% (33/48)
response v2	0	4	0	79.2% (38/48)

Prompt v2 adds “be specific, not stock” rules: mention at least two concrete details from the review, avoid stock phrases, match the emotional weight and make no assumptions. It improved acceptance from 68.8% to 79.2%. By sentiment (v2): negative 88%, positive 83%, neutral 58%. By language: German, English and Spanish 88%; French and Chinese 75%; Japanese 62%.

Important: the proposal asked for an 80% human-acceptance test. The team decided to skip it, so the 79.2% figure is a model’s opinion, not human approval. It is a useful signal (the judge caught, for example, replies that assumed the product worked), but human review remains the real test. The judge also made mistakes: it once flagged the customer’s supplied first name as an “invented fact” until we told it that the name is system-provided.

Guardrails held: no reply was in the wrong language, and prompt v2 produced no promised outcomes. Typical remaining failures are replies that assume something the review never said and empathy lines that could be pasted anywhere. The approval queue works as designed: of 48 v2 replies, 24 negative plus 7 others waited for a human and 17 were auto-approved.

6.7 Dashboard and Slack alerts

The Google Sheets dashboard has four tabs. **Reviews** holds the 1,560 analysed reviews; **Alerts** holds the detected spikes; **Data** holds formula tables for weekly sentiment, topic counts, issue categories and spike trends; **Dashboard** shows the summary figures and charts (issue-category bar chart, sentiment by week, topic share and a weekly spike line chart; see Appendix B). Review dates are simulated, so the trend charts demonstrate the detector, not real market behaviour.

Workflow 04 (nine nodes) reads the Reviews tab through the sheet’s CSV export, counts negative reviews per issue category per week, applies the same z-score rule in JavaScript and posts one summary message to a Slack incoming webhook. It found the same four spikes with identical z-scores as the Python detector (12.56, 9.80, 4.54, 4.26) and needs no Claude calls, so it costs nothing to run.

Lesson learned: the first version reported sent even though nothing had reached Slack. With the placeholder webhook URL still in place, Slack’s documentation website answered the POST with an HTML page and HTTP 200. The check now requires the exact reply ok that a real Slack webhook returns; an HTML page or an error status is recorded as failed with a hint. Judge success on the response content, not only the HTTP status.

6.8 API service and LLMOps

A FastAPI service exposes the three endpoints from the proposal plus a health check:

Endpoint	What it does
GET /health	Service status
POST /analyze	Up to 20 reviews; returns sentiment (from estimated stars), topic and confidence
POST /draft-response	Classifies if needed, writes an issue phrase for negatives, drafts the reply, optionally judges it and returns a status: negatives are always pending_human_review; others are auto_approved only if the judge accepts
POST /trends	Takes (cluster, date) events; returns spikes and the latest week-over-week table (needs at least 5 weeks)

LLMOps features: every Claude call is logged with prompt version, model, tokens, cost and latency to data/processed/llm_calls.jsonl; a hard spend cap (MAX_SPEND_USD, default \$1) makes the service

answer HTTP 429 instead of spending more; and SERVICE_API_KEY makes callers send an X-API-Key header. A live check with real Claude calls classified four multilingual reviews correctly, drafted and judged a negative English reply (issue phrase “arrived late and box was crushed”, scores 4/5/5/5, queued for a human) and auto-approved a positive Spanish reply (5/5/5/5), for a total cost of \$0.0115.

6.9 The full pipeline in n8n (workflow 03)

Workflow 03 has 25 nodes and covers the brief’s first deliverable: ingest, simulated dates, translation, sentiment and topic, issue category for negatives, reply drafting, per-review status and a summary of counts, accuracy, tokens, cost and timing. Each Claude stage retries three times and continues on error. It was run locally and on the team’s n8n Cloud with the same result.

Measure (120 reviews)	Local run	n8n Cloud run
Translation	100 ok, 20 not needed, 0 failed	100 ok, 20 not needed
Classification	120 ok	120 ok
Issue category	62 ok	60 ok, 59 not needed, 1 failed
Replies	58 draft ready + 62 pending human review	59 draft ready + 61 pending human review
3-way accuracy / positive vs negative	76.7% / 87.5%	75.8% / 87.5%
Cost	about \$0.30	\$0.2987
Wall time	about 80 s	81.3 s (0.68 s per review)

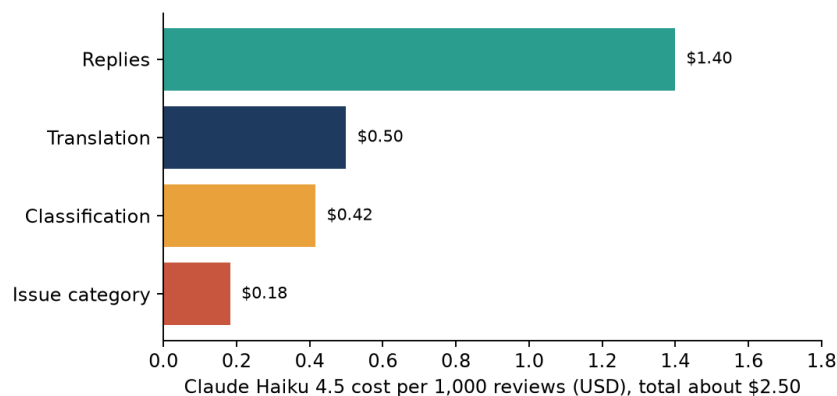


Figure 5. Where the money goes: about \$2.50 per 1,000 reviews, by stage.

6.10 Demo app and deployment

Demo app. `app.py` is a Gradio app with three tabs that reuse the code in `src/`. Saved examples: shows 13 real reviews in six languages with the stored classification, issue phrase, reply and judge verdict, and calls no model. Your own review: sends a pasted review to Claude Haiku 4.5 (classification, issue phrase, drafted reply, optional Sonnet 5 judge). Spike detection: runs the workflow 04 detector on the dashboard data, also without a model call.

Where it runs. The app is public at <https://huggingface.co/spaces/SrinivasanHF/review-insights-demo> (direct address <https://srinivasanhf-review-insights-demo.hf.space>). The Space uses Hugging Face’s free ZeroGPU tier because the free CPU option was not available to the account; the app itself uses no GPU, and a placeholder function satisfies the tier’s start-up check. The Anthropic key is stored as a Space secret and is never in the code or the repository. The live tab is limited by a spend cap (default \$0.50, counted in a log

file that restarts with the Space), 10 calls per browser session and an optional access code; an Anthropic Console spend limit is the backstop.

Deployment overview.

Part	Where it runs	How it was deployed
Workflows 01 to 04	n8n Cloud, capstoneproject-group5.app.n8n.cloud (trial to about 16 Oct 2026)	Imported from the JSON files in n8n/; Header Auth credential and Slack webhook added by the team
Demo app	Hugging Face Space SrinivasanHF/review-insights-demo	scripts/build_space.py builds a 196 KB bundle; scripts/deploy_space.py uploads it
Dashboard	Google Sheets (four tabs)	Data pasted from the repository CSVs; read by workflow 04 through its CSV export
Alerts	Slack app “Review Insights Alerts”, channel #all-capstone-project	Incoming webhook pasted into workflow 04
API service	Local (uvicorn); not public	Run from the repository; spend cap and API key supported
Code and documents	GitHub: west01srinivas-hue/Capstone_Project_Group_5	Committed from the project folder; no keys stored

Moving n8n accounts. Imported workflows never carry credentials, so on 2 October, when the team moved to a new n8n Cloud account, the four workflows were re-imported, the credential was created again and attached to the five Claude nodes, and the Slack webhook was pasted again. Verification on the new account: workflow 03 processed 120 reviews in 83 s for \$0.30 with all 120 carrying a product category, and workflow 04 posted the four spikes to Slack.

7. Methods tried, challenges and how we overcame them

Challenge	What happened	How we overcame it
Dataset loading	The Hugging Face copy uses a legacy loading script that the current datasets library refuses.	Read the per-language .jsonl files directly; later switched to the team’s Kaggle copy and verified it is identical.
Sorted data	Files are sorted by star rating, so the first N rows were all one-star reviews.	Stratified sampling: 40 reviews per star per language.
No dates or categories	MARC (HF copy) has no review date; the Kaggle copy adds category but still no date.	Joined product categories from Kaggle; simulated dates with planted spikes and said so everywhere.
Model choice	Groq’s free tier had latency of 1.5 to 58 s and no Llama chat models.	Compared on 40 reviews, then chose Claude only.
Three-way sentiment stuck near 78%	Neutral (three-star) reviews are truly mixed; labels are noisy.	Tried prompt v2/v3 and a stacker; reported the ceiling honestly and explained the star proxy.
Clusters too diffuse	HDBSCAN gave mostly noise; native-text silhouette was 0.04.	Embedded Claude’s English issue phrase and used k-means (silhouette 0.07).
Judge failures	Sonnet 5 returns a thinking block first and can run out of tokens; it flagged a supplied name as invented.	Read the first text block, raised max tokens to 2,000, told the judge the name is system-provided, re-judged all runs.
Silent n8n failures	“Continue on error” shows green even when calls fail; the execution view can show empty output for the last nodes.	Always read the summary node’s counts; run from the editor to see node output.
False “sent” in Slack workflow	Placeholder URL returned an HTML page with HTTP 200.	Require the reply body to be exactly ok; test the HTML, error and success cases locally.

Challenge	What happened	How we overcame it
Cloud limits	n8n Cloud cannot run the embedding model or read local files; the free trial is time-limited (our current account runs to about 16 October, after the final presentation).	Claude classifies into the 13 named categories inside n8n; workflows are kept as JSON in the repository so they import into any n8n.
Budget	Only \$5 of Claude credit.	Small samples, Haiku for volume, cached calls, spend cap in the API, cost per stage recorded.

8. Test cases, results and experiment log

8.1 Automated tests

`pytest tests -q` runs 16 tests and makes no Claude calls, so it costs nothing. They cover spike detection (including the week arithmetic of `/trends`), API-key authentication, input validation and the spend cap, and the demo app (saved examples, spike tab, live-tab limits).

8.2 Functional test cases

ID	Test	Expected	Observed
T1	Workflow 01: ingest MARC sample	1,200 items, 200 per language, unique ids	Passed
T2	Workflow 02: translate 120 reviews	100 translated, 20 skipped, 0 failed	Passed locally and in n8n Cloud
T3	Workflows 02 and 03 with an invalid API key	Run finishes; every review marked failed with the error	Passed; no crash
T4	Workflow 03: full pipeline, 120 reviews	No hard failures; accuracy in line with Section 6.3	Passed; 0 hard failures, 75.8% / 87.5%
T5	Workflow 04 with a fake Slack endpoint	Same four spikes and z-scores as the Python detector; one message	Passed
T6	Workflow 04, Slack endpoint returns HTTP 500	Run completes; status failed	Passed
T7	Workflow 04, placeholder URL (HTML page with HTTP 200)	Status failed, not sent	Failed at first, fixed, passed
T8	Workflow 04 against real Slack	Message appears in the channel; status sent	Passed (four alerts posted)
T9	API: request without the API key, invalid input, spend cap reached	Rejected; validation error; HTTP 429	Passed (automated tests)
T10	API live check: 4 multilingual reviews, 2 drafts	Correct labels; negative queued for a human	Passed, cost \$0.0115

8.3 Experiment log summary

The full log with tables and reasoning is `docs/experiment_log.md` in the repository. In brief:

No.	Experiment	Main finding
E1	Data source and sampling	Stratified 1,200-review sample; Kaggle and Hugging Face copies identical
E2	Ingestion workflow	1,200 items, balanced by language and star
E3	Translation model comparison	Groq and Claude both 40/40; Claude chosen; \$0.54 per 1,000
E4	Translation workflow	0 failures; failure path handled
E5	Sentiment classification	Claude 77.3% 3-way, 88.5% two-way vs baseline 72.9% / 83.9%
E6	Lifting the neutral class	Best 78.1%; stacker adds nothing; star MAE 0.47

No.	Experiment	Main finding
E7	Issue clustering	k = 13; issue phrases beat native text (0.07 vs 0.04)
E8	Trend detection	Precision 88.6%, recall 94.3%, about 0.5 false alarms per 12 weeks
E9	Response generation	Judge-acceptable 68.8% to 79.2% (v1 to v2); human acceptance not measured
E10	API service and LLMOps	3 endpoints, 11 tests, spend cap, call logging
E11	Full pipeline in n8n	\$0.30 per 120 reviews, 81 s, no hard failures
E12	Dashboard and Slack alert	Four spikes posted; success check fixed
E13	Demo app on a Hugging Face Space	Three-tab Gradio app, live and spend-capped; all tabs tested on the Space
E14	Product category in n8n	120 of 120 reviews carry a category (lookup from a CSV in the repository)
E15	Move to a new n8n Cloud account	Workflows 03 and 04 re-verified end to end on 2 Oct 2026

8.4 Claude spend

Item	Cost (USD)
Translation tests	0.14
Sentiment experiments (E5 and E6)	1.18
Issue phrases and cluster labels	0.23
Response generation and judging	0.61
API test, issue-category check, workflow 03 local test	0.34
Total recorded	about 2.51 of 5.00

The calls logged above add up to about \$2.51. Runs on n8n Cloud (including three whose cost was not recorded), the demo app and the final test runs make up the rest. The Anthropic Console shows a total of \$4.29 for the whole project, within the \$5 budget.

9. Demo plan

The demo goes back to the four situations in Section 2. Tasks get harder, so the audience sees the easy case work and then the cases that used to be a problem.

Level	Task	What the audience sees	Problem it answers
1 Easy	Classify one English review: “arrived late and the box was crushed”.	Negative, topic shipping, issue phrase and estimated stars, in a few seconds.	Late, generic handling of one review
2 Medium	Send a Japanese neutral review through translation, classification and drafting.	English translation, mixed label, reply written in Japanese within the guardrails.	Language barrier
3 Hard	Run n8n workflow 03 on a mixed batch of reviews in all six languages.	Summary node: counts per stage, accuracy, cost, time; negatives waiting in the approval queue.	Manual reading at volume
4 Hard	Show the mixed three-star review “I love the sound but it turns itself back on”.	Estimated stars and label, a specific reply that names the fault (run it before the demo to confirm the output), and a note on why three-way accuracy is limited.	Stars hide the reason
5 Hardest	Run workflow 04 on the dashboard sheet.	Detected spikes, week-over-week delta and a live Slack message linking to the dashboard.	Spikes found late

Backup plan: keep recorded outputs and the exported n8n workflow JSON files in case the network or the n8n trial fails on the day. Our current n8n Cloud trial runs to about 16 October, after the presentation, and the workflows are also kept as JSON so they can be imported into another n8n (or run locally with `npx n8n`) beforehand.

10. How to run the application

10.1 Setup (Windows)

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env      # then fill in ANTHROPIC_API_KEY
python scripts/check_keys.py # tests the Claude key
```

Set KAGGLE_MARC_DIR in .env to the folder that holds train.csv, validation.csv and test.csv from the Kaggle dataset mexwell/amazon-reviews-multi. The sample CSVs in data/samples/ are already in the repository.

10.2 n8n workflows

1. In n8n choose Workflows, Create workflow, the ... menu, then **Import from file**, and pick a file from the n8n/ folder.
2. Create the Claude credential once: Credentials, Create, **Header Auth**, Name x-api-key, Value = your Anthropic API key. Open each Claude HTTP node and select that credential.
3. For workflow 04, share the dashboard sheet as “Anyone with the link can view” and paste your Slack incoming-webhook URL into the **Post to Slack** node, then save.
4. Click **Execute workflow**. Read the summary node (Pipeline summary or Alert result) rather than trusting the green ticks.

File	Purpose	Approximate cost
01_ingestion_marc_to_unified_schema.json	Ingest and sample reviews	Free
02_ingestion_and_translation.json	Adds Claude translation	About \$0.06 at the default size
03_full_pipeline.json	Full pipeline; perStar in Config sets the size	About \$0.30 for 120 reviews; \$3 at 40 per star
04_trend_detection_slack_alert.json	Spike detection and Slack alert	Free

10.3 Analysis scripts

```
python scripts/eval_sentiment.py          # sentiment experiments (cached calls)
python scripts/run_stage4.py issues        # issue phrases
python scripts/run_stage4.py cluster issue # k-means clustering
python scripts/run_stage4.py label issue   # cluster labels
python scripts/run_stage4.py trends issue  # trend detection
python scripts/eval_responses.py          # replies + judge (RESPONSE_VERSION=v1|v2)
```

10.4 API service and tests

```
set SERVICE_API_KEY=choose-a-key # optional; callers then send X-API-Key
set MAX_SPEND_USD=1.0             # hard cap on Claude spend
uvicorn src.api.main:app --port 8000
# interactive docs: http://localhost:8000/docs
pytest tests -q                   # 16 tests, no Claude calls
```

10.5 Demo app and Hugging Face Space

```

pip install gradio # once; the live tab needs ANTHROPIC_API_KEY in .env
python app.py # http://127.0.0.1:7860
python scripts/build_space.py # builds space_build/ (about 200 KB, no keys)
python scripts/deploy_space.py # uploads it (HF_TOKEN with write access in .env)

```

To publish your own copy: create a Space on huggingface.co (SDK Gradio), add the secret ANTHROPIC_API_KEY yourself in Settings, Variables and secrets, and run the two scripts above. Delete the write token when you are done deploying.

10.6 Repository map

Path	Contents
src/analysis, src/response, src/api, src/llmops	Clustering and trend detection; reply generation and judge; FastAPI service; call tracking and spend cap
prompts/	Versioned prompts: translate, classify v1 to v3, issue phrase, issue category and taxonomy, judge, reply prompts and 21 playbooks
n8n/	The four exported workflows
scripts/	Data sampling, evaluation and stage scripts
data/samples/	Review samples, dashboard data and alert list
docs/experiment_log.md	The full experiment log
tests/	The 16 pytest tests
app.py, space/, scripts/build_space.py, scripts/deploy_space.py	Gradio demo app and the Hugging Face Space bundle and upload scripts

11. Limitations

- **Simulated dates.** The dataset has no review dates, so trend detection was tested on simulated weeks with planted spikes, and the threshold was tuned on the same simulation. Real data may behave differently.
- **Star-derived labels.** Sentiment accuracy is measured against a noisy proxy. Three-way accuracy is 78.1%, below the 85% target, and the neutral class is recognised only about a third of the time.
- **No human acceptance test.** Reply quality is judged by a model. The 79.2% figure is not human approval.
- **Weak spots in replies.** Neutral reviews (58% accepted) and Japanese (62%) are the weakest; the reply signature stays in English in other languages; some replies assume facts the review did not state.
- **Issue categories in n8n.** A Claude classifier over 13 fixed categories replaces the embedding step, and it agrees with k-means only 43% of the time. The categories come from one sample and would need refreshing on a real feed.
- **Small evaluation sets.** 48 replies and 960 test reviews are enough to show trends, but small differences (for example between prompt v1 and v2 on the development split) are within noise.
- **Not yet automatic.** Results reach the Google Sheet by paste or import, and the n8n workflows are started by hand. Only the demo app is public (Hugging Face Space); the API service runs locally.
- **Trial and budget.** The n8n Cloud trial is time-limited (we moved to a new account on 2 October, valid to about 16 October), and Claude credit limits how many full runs we can afford. We measured throughput on 120 reviews and extrapolated to 1,000.

- **Placeholders.** The brand “Acme Store”, its support address and the reviewer names are placeholders.

12. Future scope and scaling

- **Real data feed.** Connect live review sources (marketplace exports, app-store and social feeds) with real timestamps, then retune the spike threshold on genuine weeks.
- **Human evaluation.** Run the reply acceptance test with support staff (the review sheet and scoring script already exist) and use the results to improve the playbooks.
- **Better neutral handling.** Ask for a 1 to 5 rating plus a separate “mixed” flag, or fine-tune a multilingual model on labelled neutral reviews.
- **Self-updating dashboard.** Append pipeline results to Google Sheets or a database from workflow 03 and build Looker Studio pages on top.
- **Scaling.** Use batch and prompt-caching features of the model API to cut the \$2.50 per 1,000 reviews, run the embedding step in a small service, and move the API to a hosted environment with proper tracing (for example Langfuse).
- **More languages and channels.** Extend beyond six languages, add Slack or email delivery of approved replies, and track how long human approval takes to measure the 3-hour goal properly.
- **Closed loop.** Feed approved and edited replies back as examples, and report which issues shrink after a fix.

13. Business perspective

Cost. The measured cost is about \$2.50 per 1,000 reviews with Claude Haiku 4.5 (replies \$1.40, translation \$0.50, classification \$0.42, issue category \$0.18). At 1,000 reviews per day this is about \$75 per month of model usage. Trend detection and the Slack alert cost nothing to run.

Speed. The automated part takes about 11 minutes per 1,000 reviews, so a day’s feed is analysed in one short run instead of a working day of reading. A draft reply exists about a minute after a batch starts; the remaining time to reply is the human approval of negative drafts, which we did not measure.

Illustration of the saving

The table below is an illustration, not a measurement. The manual time per review is an assumption that the team should replace with its own figure.

	Manual handling	With the pipeline
Reading, translating and drafting a reply for 1,000 reviews	About 33 staff hours if a review takes 2 minutes (assumption)	About 11 minutes of automated processing
Human effort remaining	All of it	Approving and editing drafts, mainly for negative reviews (our balanced sample is about 49% negative; a real feed usually has fewer)
Cost per 1,000 reviews	Staff time	About \$2.50 of model usage plus approval time
Spike discovery	In the next periodic report	Next scheduled run of workflow 04, with a Slack message

Business value. Faster and more consistent replies in the customer’s language, earlier warning about delivery, packaging or price problems, and a shared view of the reasons behind ratings. The human

approval queue keeps a person in charge of anything sensitive, and the spend cap and call log make the cost predictable.

Risks and how we handle them. Wrong or over-promising replies (guardrails, judge, mandatory human approval for negatives); privacy (the pipeline asks for no personal data and drafts use first names only); model changes or price changes (versioned prompts, model name in configuration); overreliance on the automated sentiment (report the neutral-class weakness, offer a borderline-review flag).

14. Conclusion

We built a complete, working pipeline for multilingual review analysis and reply drafting, verified it on real reviews in six languages, and connected it to a dashboard and Slack. Positive-versus-negative accuracy of 88.4% and a cost of about \$2.50 per 1,000 reviews show that the approach is practical. The results that fall short of the proposal (three-way accuracy of 78.1%, star error of 0.47 and reply acceptance measured by a model) are explained rather than hidden. The most valuable next steps are testing on a real dated feed and asking people to review the drafts.

References

1. Keung, P., Lu, Y., Szarvas, G., & Smith, N. A. (2020). The Multilingual Amazon Reviews Corpus. Proceedings of EMNLP 2020.
2. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019. (Model used: paraphrase-multilingual-MiniLM-L12-v2.)
3. Rahman et al. (2025), Scientific Reports, multilingual sentiment with XLM-RoBERTa-style models, as cited in the team's project proposal.
4. Anthropic. Claude API documentation and model pricing (Haiku 4.5 and Sonnet 5).
5. n8n documentation: HTTP Request node, Code node, error handling. Slack documentation: Incoming Webhooks.

Appendix A. Prompts and versions

File	Purpose	Status
translate_v1.txt	Batch translation and language detection	Used
classify_v1.txt, classify_v2.txt	Sentiment and topic	Compared on the dev split; v2 better on two-way accuracy
classify_v3.txt	Adds estimated stars	Production choice (stars mapped to label)
issue_v1.txt	Short English issue phrase for negatives	Used for clustering
issue_cluster_v1.txt, issue_taxonomy_v1.json	Assign a negative review to one of 13 categories	Used in n8n
response/system_v1.txt, system_v2.txt	Reply drafting with guardrails	v2 better (79.2% vs 68.8% judge-acceptable)
response/playbooks_v1.json	21 playbooks (3 sentiments x 7 topics)	Used
judge_v1.txt	Sonnet 5 scoring rubric and flags	Used

Appendix B. Dashboard screenshots

The Dashboard tab of the Google Sheet holds the summary figures and four charts: the issue-category bar chart, sentiment by week, topic share, and the weekly spike line chart. The Alerts tab lists the four spikes reported in Section 6.5. Review dates are simulated.

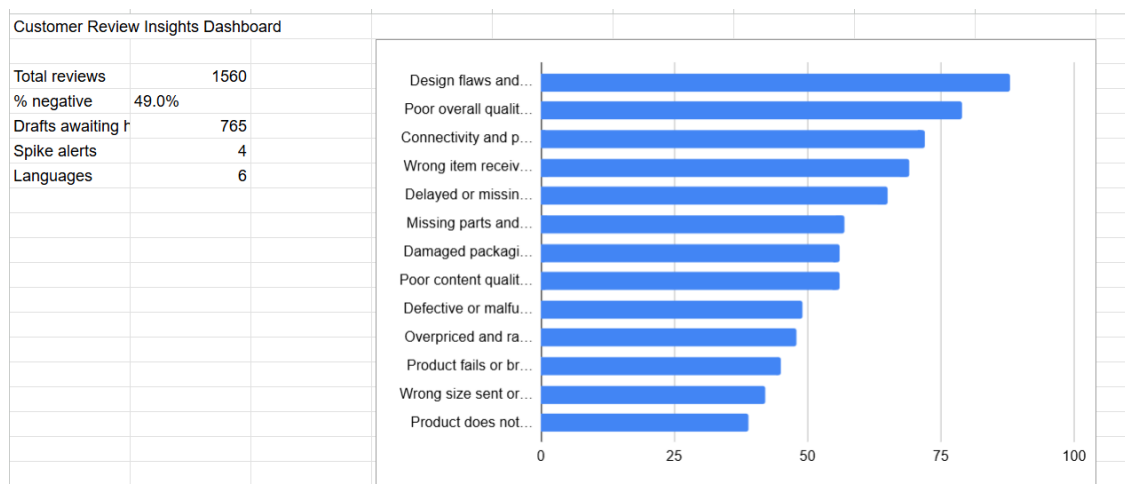


Figure B1. Dashboard summary figures (1,560 reviews, 49.0% negative, 765 drafts awaiting human review, 4 spike alerts, 6 languages) and negative reviews per issue category.

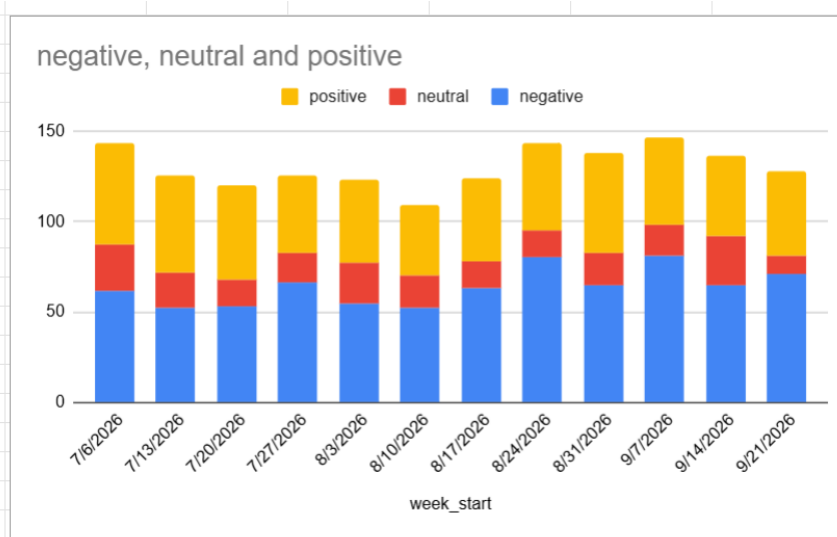


Figure B2. Reviews per week by sentiment (positive, neutral, negative) for the 12 simulated weeks.

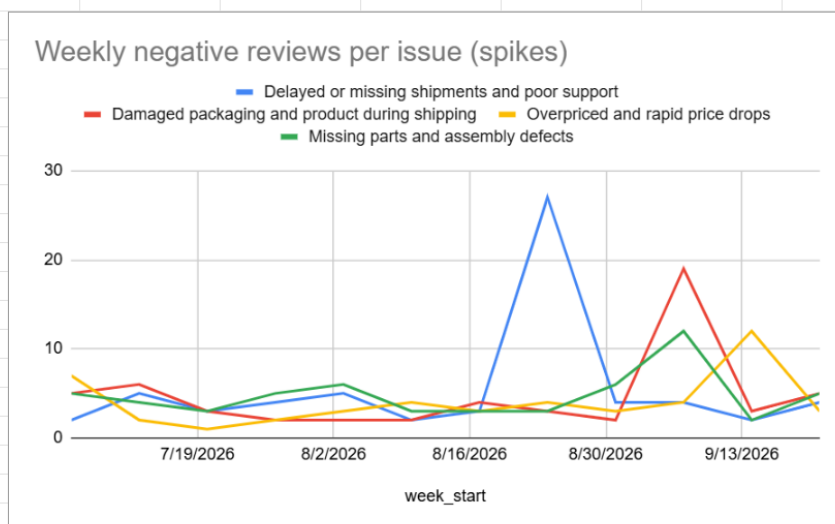


Figure B3. Weekly negative reviews for the four issue categories that raised alerts (Data tab range L1:P13).

The delayed-shipments spike (27 reviews) is in the week of 24 August, damaged packaging (19) and missing parts (12) in the week of 7 September, and price complaints (12) in the week of 14 September.

 **Review Insights Alerts** APP 3:49 PM
 Negative review spike detected (4 issues)

- Delayed or missing shipments and poor support: 27 negative reviews in the week of 2026-08-24 (baseline 3.5, z 12.56, +24 vs previous week)
- Damaged packaging and product during shipping: 19 negative reviews in the week of 2026-09-07 (baseline 2.75, z 9.8, +17 vs previous week)
- Overpriced and rapid price drops: 12 negative reviews in the week of 2026-09-14 (baseline 3.5, z 4.54, +8 vs previous week)
- Missing parts and assembly defects: 12 negative reviews in the week of 2026-09-07 (baseline 3.75, z 4.26, +6 vs previous week)

[Open the Review Insights Dashboard](#)

Figure B4. The Slack alert posted to #all-capstone-project by workflow 04, listing the four detected spikes with counts, baseline and z-score.

Optional: add the topic-share pie chart from the Dashboard tab.